Spring MVC 購物車完整教學文件

> **技術棧 (Tech Stack)**：Maven + Spring WebMVC + Hibernate + MySQL → Spring Boot + Thymeleaf + Spring Data JPA **適合程度**：有 Java 基礎，初學 Spring 框架者

目錄

1. 系統架構概覽
2. Part 1：Maven + Spring WebMVC + Hibernate
   - 步驟 1：建立 Maven Webapp
   - 步驟 2：加入 Maven 依賴
   - 步驟 3：設定 Spring WebMVC
   - 步驟 4：測試 Spring WebMVC
   - 步驟 5：建立資料模型
   - 步驟 6：建立控制器
   - 步驟 7：建立 JSP 檢視畫面
   - 步驟 8：整合 Hibernate 與 MySQL
3. Part 2：Spring Boot + Thymeleaf 版本
   - 資料庫設定
   - Entity 與 Repository
   - 控制器與畫面
   - 購物車功能
4. 兩版本架構比較
5. 常見錯誤與陷阱
6. 練習題

系統架構概覽

本專案實作一個簡易購物車 (Shopping Cart) 系統，涵蓋兩種實作方式：

```
Part 1 架構 (傳統 Spring MVC)
┌──────────┐    HTTP    ┌──────────────────┐    Model    ┌──────────────┐
│  Browser │ ──────────→ │ DispatcherServlet │ ──────────→ │ Controller │
└──────────┘            │ (Front Controller)│             └──────────────┘
                        └──────────────────┘                     │
                                                                 ▼
                                                        ┌──────────────┐
                                                        │  Service /   │
                                                        │ ProductModel │
                                                        └──────────────┘
                                                                 │
                                                                 ▼
                                                        ┌──────────────┐
                                                        │  Hibernate   │
                                                        │ (ORM 層)     │
                                                        └──────────────┘
                                                                 │
```

```
                                                    ┌──────▼──────┐
                                                    │   MySQL     │
                                                    └─────────────┘

Part 2 架構 (Spring Boot + JPA)
Controller → Spring Data JPA Repository → MySQL (自動實作 CRUD)
```

購物車核心邏輯：商品資料存於 DB，加入購物車後存入 `HttpSession`，同一瀏覽器連線期間持久保存。

Part 1：Maven + Spring WebMVC + Hibernate

步驟 1：建立 Maven Webapp

建立標準 Maven Web 專案後，將 `web.xml` 升級至 **Servlet 4.0** 規格：

```xml
<!-- web.xml -->
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
    id="WebApp_ID" version="4.0">

  <display-name>mvsusertable</display-name>
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
</web-app>
```

> **重點**：`version="4.0"` 對應 Tomcat 9+，確保與 Spring 5 相容。

步驟 2：加入 Maven 依賴

`pom.xml` 中加入以下 5 個核心依賴：

```xml
<!-- JSTL：JSP 標準標籤庫，用於 forEach、if 等 -->
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>jstl</artifactId>
    <version>1.2</version>
</dependency>

<!-- MySQL 驅動程式：連接 MySQL 8 資料庫 -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.33</version>
```

```
</dependency>

<!-- Spring WebMVC: 核心 MVC 框架 -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>5.3.39</version>
    <scope>compile</scope>
</dependency>

<!-- Hibernate: ORM (物件關聯對映) 框架 -->
 <dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>5.6.15.Final</version>
    <scope>compile</scope>
</dependency>

<!-- Gson: Java 物件 ⇔ JSON 轉換工具 -->
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
</dependency>
```

依賴	用途
`jstl`	JSP 頁面的邏輯標籤（迴圈、條件）
`mysql-connector-java`	JDBC 連接 MySQL
`spring-webmvc`	MVC 控制器、路由、視圖解析
`hibernate-core`	ORM，Java 類別對應資料庫表格
`gson`	JSON 序列化（本專案備用）

步驟 3：設定 Spring WebMVC

3a：在 `web.xml` 中註冊 DispatcherServlet（前端控制器）

```
<!-- web.xml — 加入以下 servlet 設定 -->
<servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-
class>
    <!-- load-on-startup=1 代表伺服器啟動時就初始化，不等第一個請求 -->
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
```

```
    <!-- / 代表攔截所有 HTTP 請求（除了 .jsp） -->
    <url-pattern>/</url-pattern>
</servlet-mapping>
```

3b：建立 `dispatcher-servlet.xml`（放在 `WEB-INF/` 下）

```
<?xml version='1.0' encoding='UTF-8' ?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:p="http://www.springframework.org/schema/p"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
               http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
           http://www.springframework.org/schema/mvc
               http://www.springframework.org/schema/mvc/spring-mvc-4.0.xsd
           http://www.springframework.org/schema/context
               http://www.springframework.org/schema/context/spring-context-
4.0.xsd">

    <!-- 啟用 @Controller、@RequestMapping 等 Annotation 支援 -->
    <mvc:annotation-driven />

    <!-- 靜態資源直接存取，不經過 DispatcherServlet -->
    <mvc:resources location="/images/" mapping="/images/**" />
    <mvc:resources location="/js/"     mapping="/js/**" />

    <!-- 掃描 demo.controller 套件，自動偵測 @Controller -->
    <context:component-scan base-package="demo.controller" />

    <!-- 視圖解析器：Controller 回傳 "index" → /WEB-INF/jsp/index.jsp -->
    <bean id="viewResolver"

class="org.springframework.web.servlet.view.InternalResourceViewResolver"
          p:prefix="/WEB-INF/jsp/"
          p:suffix=".jsp" />
</beans>
```

> **概念：視圖解析器 (ViewResolver)** Controller 只回傳邏輯視圖名稱（如 `"index"`），ViewResolver 負責轉換成實際路徑 `/WEB-INF/jsp/index.jsp`，讓 Controller 不需知道頁面放在哪裡。

步驟 4：測試 Spring WebMVC

建立第一個控制器確認框架運作正常：

```
package demo.controller;
```

```java
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.ResponseBody;

@Controller  // 宣告此類別為 Spring MVC 控制器
public class HelloWorldController {

    // redirect 範例: 接到 /redirect 後, 重導到 /hello
    @RequestMapping(value = "/redirect", method = RequestMethod.GET)
    public String authorInfo(Model model) {
        return "redirect:/hello";  // redirect: 前綴觸發 HTTP 302 重導
    }

    // 一般視圖回傳: 將 "greeting" 資料傳給 helloworld.jsp
    @RequestMapping("/hello")
    public String hello(Model model) {
        model.addAttribute("greeting", "Hello Spring MVC");
        return "helloworld";  // → 解析為 /WEB-INF/jsp/helloworld.jsp
    }

    // @ResponseBody: 直接回傳字串給瀏覽器, 不解析視圖
    @RequestMapping(value = "/text", produces = "text/plain;charset=utf-8")
    @ResponseBody
    public String authorInfo2(Model model) {
        return "ResponseBody";
    }
}
```

對應的 `helloworld.jsp`:

```jsp
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head><title>Spring MVC - Hello World</title></head>
<body>
    <!-- ${greeting} 使用 EL (Expression Language) 取得 Model 中的資料 -->
    <h1>${greeting}</h1>
</body>
</html>
```

測試路徑:

- `http://localhost:8080/app/hello` → 顯示 "Hello Spring MVC"
- `http://localhost:8080/app/redirect` → 重導到 `/hello`
- `http://localhost:8080/app/text` → 純文字 "ResponseBody"

步驟 5：建立資料模型

購物車由三個 Model 類別組成：

`Product.java` — 商品

```
package model;

public class Product {
    private String id;      // 商品編號，如 "p01"
    private String name;    // 商品名稱
    private String photo;   // 圖片路徑，如 "images/p1.jpg"
    private double price;   // 價格

    // 無參數建構子（Hibernate 反射時需要）
    public Product() {}

    // 全參數建構子（方便初始化假資料）
    public Product(String id, String name, String photo, double price) {
        this.id = id;
        this.name = name;
        this.photo = photo;
        this.price = price;
    }

    // Getter / Setter（省略，標準 JavaBean 格式）
    // getId(), setId(), getName(), setName(), getPhoto(), setPhoto(), getPrice(),
setPrice()
}
```

`Item.java` — 購物車項目（商品 + 數量）

```
package model;

// Item 是購物車中的一筆記錄：「哪個商品」買了「幾件」
public class Item {
    private Product product;  // 關聯的商品物件
    private int quantity;     // 購買數量

    public Item() {}

    public Item(Product product, int quantity) {
        this.product = product;
        this.quantity = quantity;
    }

    // Getter / Setter 省略
}
```

> **設計模式**：`Item` 是 **Composite（組合）**，把商品與數量封裝在一起，方便計算小計（`product.price × quantity`）。

`ProductModel.java` **— 商品資料存取層**

```
package model;

import java.util.ArrayList;
import java.util.List;

// ProductModel 負責提供商品資料（初期用假資料，後期換成 DB）
public class ProductModel {
    private List<Product> products;

    public ProductModel() {
        // 初始化假資料（整合 Hibernate 後會替換這段）
        this.products = new ArrayList<Product>();
        this.products.add(new Product("p01", "JBud Elite",   "images/p1.jpg",
20));
        this.products.add(new Product("p02", "EdiMax Wifi",  "images/p2.jpg",
21));
        this.products.add(new Product("p03", "Asus Laptop",  "images/p3.jpg",
22));
    }

    // 取得全部商品
    public List<Product> findAll() {
        return this.products;
    }

    // 依 id 查詢單一商品
    public Product find(String id) {
        for (Product product : this.products) {
            if (product.getId().equalsIgnoreCase(id)) {
                return product;
            }
        }
        return null;  // 找不到回傳 null
    }
}
```

步驟 6：建立控制器

`ProductCartController.java` **— 商品列表頁**

```
package demo.controller;

import org.springframework.stereotype.Controller;
```

```java
import org.springframework.ui.ModelMap;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import model.ProductModel;

@Controller
@RequestMapping(value = "/productcart")   // 類別層級的 URL 前綴
public class ProductCartController {

    // GET /productcart → 載入商品列表，傳給 index.jsp
    @RequestMapping(method = RequestMethod.GET)
    public String index(ModelMap modelMap) {
        ProductModel productModel = new ProductModel();
        modelMap.put("products", productModel.findAll());  // 傳資料到視圖
        return "index";  // → /WEB-INF/jsp/index.jsp
    }
}
```

`CartController.java` — **購物車操作**

```java
package demo.controller;

import java.util.ArrayList;
import java.util.List;
import javax.servlet.http.HttpSession;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.*;
import model.*;

@Controller
@RequestMapping(value = "/cart")
public class CartController {

    // GET /cart/index → 顯示購物車頁面
    @RequestMapping(value = "index", method = RequestMethod.GET)
    public String index() {
        return "cart/cartindex";  // → /WEB-INF/jsp/cart/cartindex.jsp
    }

    // GET /cart/buy/{id} → 加入商品到購物車
    @RequestMapping(value = "buy/{id}", method = RequestMethod.GET)
    public String buy(@PathVariable("id") String id, HttpSession session) {
        ProductModel productModel = new ProductModel();

        if (session.getAttribute("cart") == null) {
            // 購物車不存在：建立新購物車並加入商品
            List<Item> cart = new ArrayList<Item>();
            cart.add(new Item(productModel.find(id), 1));
            session.setAttribute("cart", cart);
        } else {
            // 購物車已存在：檢查商品是否已在車內
```

```
            List<Item> cart = (List<Item>) session.getAttribute("cart");
            int index = this.exists(id, cart);

            if (index == -1) {
                // 商品不在車內：新增
                cart.add(new Item(productModel.find(id), 1));
            } else {
                // 商品已在車內：數量 +1
                int quantity = cart.get(index).getQuantity() + 1;
                cart.get(index).setQuantity(quantity);
            }
            session.setAttribute("cart", cart);
        }
        return "redirect:/cart/index";  // 加完後重導到購物車頁
    }

    // GET /cart/remove/{id} → 從購物車移除商品
    @RequestMapping(value = "remove/{id}", method = RequestMethod.GET)
    public String remove(@PathVariable("id") String id, HttpSession session) {
        List<Item> cart = (List<Item>) session.getAttribute("cart");
        int index = this.exists(id, cart);
        cart.remove(index);
        session.setAttribute("cart", cart);
        return "redirect:/cart/index";
    }

    // 輔助方法：在購物車中尋找指定 id 的商品，回傳索引（找不到回傳 -1）
    private int exists(String id, List<Item> cart) {
        for (int i = 0; i < cart.size(); i++) {
            if (cart.get(i).getProduct().getId().equalsIgnoreCase(id)) {
                return i;
            }
        }
        return -1;
    }
}
```

購物車核心邏輯流程圖：

```
GET /cart/buy/{id}
        │
        ▼
session["cart"] == null ?
    ├─ 是 → 建立新 List<Item>，加入商品(qty=1)，存入 session
    └─ 否 → 取得現有 cart
                │
                ▼
            exists(id, cart) == -1 ?
                ├─ 是 → 加入新 Item(qty=1)
                └─ 否 → 找到該 Item，quantity++
        │
```

```
        ▼
redirect:/cart/index (顯示最新購物車)
```

步驟 7：建立 JSP 檢視畫面

`index.jsp` — 商品列表

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html>
<head>
    <title>Product Page</title>
</head>
<body>
    <h3>Products Page</h3>
    <table border="1" cellpadding="2" cellspacing="2">
        <tr>
            <th>Id</th><th>Name</th><th>Photo</th><th>Price</th><th>Buy</th>
        </tr>

        <%-- c:forEach 迭代 Controller 傳來的 products 集合 --%>
        <c:forEach var="product" items="${products}">
            <tr>
                <td>${product.id}</td>
                <td>${product.name}</td>
                <%-- pageContext.request.contextPath 取得應用程式根路徑，避免路徑錯誤 --%>
                <td><img src="${pageContext.request.contextPath}/${product.photo}" width="150"></td>
                <td>${product.price}</td>
                <td>
                    <a href="${pageContext.request.contextPath}/cart/buy/${product.id}">Buy Now</a>
                </td>
            </tr>
        </c:forEach>
    </table>
</body>
</html>
```

`cartindex.jsp` — 購物車頁面

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<html>
```

```
<head><title>Cart Page</title></head>
<body>
    <h3>Cart Page</h3>
    <table border="1" cellpadding="2" cellspacing="2">
        <tr>
            <th>Option</th><th>Id</th><th>Name</th>
            <th>Photo</th><th>Price</th><th>Quantity</th><th>Sub Total</th>
        </tr>

        <%-- 宣告 total 變數，用來累計總金額 --%>
        <c:set var="total" value="0" />

        <%-- sessionScope.cart：從 HttpSession 讀取購物車資料 --%>
        <c:forEach var="item" items="${sessionScope.cart}">
            <%-- 每次迭代累加小計 --%>
            <c:set var="total" value="${total + item.product.price *
item.quantity}" />
            <tr>
                <td>
                    <%-- 刪除前彈出確認視窗 --%>
                    <a
href="${pageContext.request.contextPath}/cart/remove/${item.product.id}"
                       onclick="return confirm('確定要刪除
${item.product.name}?')">Remove</a>
                </td>
                <td>${item.product.id}</td>
                <td>${item.product.name}</td>
                <td><img
src="${pageContext.request.contextPath}/${item.product.photo}" width="150"></td>
                <td>${item.product.price}</td>
                <td>${item.quantity}</td>
                <td>${item.product.price * item.quantity}</td>
            </tr>
        </c:forEach>
        <tr>
            <td colspan="6" align="right"><strong>Sum</strong></td>
            <td>${total}</td>
        </tr>
    </table>
    <br>
    <a href="${pageContext.request.contextPath}/productcart">Continue Shopping</a>
</body>
</html>
```

步驟 8：整合 Hibernate 與 MySQL

8a：建立資料表

```
-- 建立 mvproduct 資料表
CREATE TABLE `classicmodels`.`mvproduct` (
```

```
    `id`    VARCHAR(10)    NOT NULL,
    `name`  VARCHAR(80)    NOT NULL,
    `photo` VARCHAR(255)   NOT NULL,
    `price` DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY (`id`)
) ENGINE = InnoDB DEFAULT CHARACTER SET = utf8;

-- 插入測試資料
INSERT INTO `classicmodels`.`mvproduct` VALUES ('p01', 'JBud Ear Phone',
'images/p1.jpg',  990.00);
INSERT INTO `classicmodels`.`mvproduct` VALUES ('p02', 'EDiMax Wifi',
'images/p2.jpg', 1200.00);
INSERT INTO `classicmodels`.`mvproduct` VALUES ('p03', 'ASUS Computer',
'images/p3.jpg', 25000.00);
```

8b：建立 `hibernate.cfg.xml`（放在 `src/main/java/`）

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- JDBC 驅動程式 -->
        <property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <!-- 連線 URL: serverTimezone 必填, 否則 MySQL 8 會報錯 -->
        <property name="hibernate.connection.url">
            jdbc:mysql://localhost:3306/classicmodels?
serverTimezone=CST&amp;useUnicode=true&amp;characterEncoding=utf8
        </property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">1234</property>
        <!-- 告訴 Hibernate 使用 MySQL 的 SQL 方言 -->
        <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
        <!-- 對應 Entity 類別 (加入 Hibernate 後必須宣告)  -->
        <mapping class="model.Product"/>
    </session-factory>
</hibernate-configuration>
```

8c：為 `Product.java` 加入 JPA 標註（Annotation）

```
package model;

import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
```

```java
@Entity                 // 宣告此類別對應一張資料庫表格
@Table(name = "mvproduct")  // 指定表格名稱（預設用類別名稱）
public class Product {

    @Id                 // 標記主鍵（Primary Key）欄位
    private String id;
    private String name;    // 欄位名稱預設對應同名的 DB 欄位
    private String photo;
    private double price;

    // 建構子與 Getter/Setter 同前
}
```

8d：修改 `ProductModel` 改從 DB 讀取資料

```java
package model;

import org.hibernate.*;
import org.hibernate.cfg.Configuration;
import java.util.*;

public class ProductModel {
    private List<Product> products;

    public ProductModel() {
        from_mvDB();  // 改為從資料庫載入
    }

    private void from_mvDB() {
        // 1. 建立 Hibernate 設定物件
        Configuration configObj = new Configuration();
        configObj.addAnnotatedClass(model.Product.class);  // 註冊 Entity
        configObj.configure("hibernate.cfg.xml");          // 載入設定檔

        // 2. 開啟 Session（對應一次 DB 連線）
        Session session = configObj.buildSessionFactory().openSession();
        Transaction tx = null;
        try {
            tx = session.beginTransaction();
            // 3. 使用 HQL（Hibernate Query Language）查詢
            //    HQL 使用類別名稱，不是資料表名稱
            products = session.createQuery("FROM Product").list();
            tx.commit();
        } catch (HibernateException e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        } finally {
            session.close();  // 4. 必須關閉 Session 釋放連線
        }
    }
```

```java
    public List<Product> findAll() { return this.products; }

    public Product find(String id) {
        for (Product p : products) {
            if (p.getId().equalsIgnoreCase(id)) return p;
        }
        return null;
    }
}
```

> **注意**：每次建立 `ProductModel` 都會建立新的 `SessionFactory`，在生產環境中效能較差。建議使用 Spring 的 `@Bean` 注入共享的 `SessionFactory`。

Part 2：Spring Boot + Thymeleaf 版本

Spring Boot 大幅簡化設定，以下為現代化重構版本。

資料庫設定

```properties
# src/main/resources/application.properties

# MySQL 連線設定
spring.datasource.url=jdbc:mysql://localhost/demo
spring.datasource.username=root
spring.datasource.password=1234
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate 設定
spring.jpa.show-sql=true                              # 在 console 顯示 SQL
spring.jpa.properties.hibernate.format_sql=true       # 格式化 SQL 輸出
spring.jpa.properties.hibernate.hbm2ddl.auto=update   # 自動更新資料表結構
```

> `hbm2ddl.auto` **選項說明**
>
> | 值 | 行為 |
> | --- | --- |
> | `update` | 自動新增缺少的欄位，不刪除資料 |
> | `create` | 每次啟動重建表格（開發初期） |
> | `validate` | 只驗證 schema，不修改（生產環境） |
> | `none` | 不做任何 schema 操作 |

Entity 與 Repository

`Product.java` **（使用 Lombok 簡化）**

```java
import javax.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "products")
@Data            // Lombok: 自動產生 getter、setter、equals、hashCode、toString
@NoArgsConstructor   // Lombok: 自動產生無參數建構子
@AllArgsConstructor  // Lombok: 自動產生全參數建構子
public class Product {
    @Id
    Integer id;
    String productname;
    String description;
    int price;
    String path;       // 圖片路徑
}
```

> **Lombok 優點**：省去大量樣板程式碼（Boilerplate），讓類別更簡潔。

ProductRepository.java

```java
import org.springframework.data.jpa.repository.JpaRepository;

// JpaRepository<Entity類別, 主鍵型別>
// Spring Data JPA 自動實作: findAll(), findById(), save(), deleteById()...
public interface ProductRepository extends JpaRepository<Product, Integer> {
    // 無需寫任何程式碼, Spring Data JPA 自動生成 CRUD 方法
}
```

控制器與畫面

ProductController.java

```java
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.ModelAndView;
import com.example.demo.model.*;

@RestController
public class ProductController {

    @Autowired
    ProductRepository repository;  // 注入 Repository (自動實作的 CRUD)
```

```
    @RequestMapping(value = "/products", method = RequestMethod.GET)
    public ModelAndView getAllProducts() {
        List<Product> data = repository.findAll();  // 查詢全部商品
        ModelAndView viewModel = new ModelAndView("productView");
        viewModel.addObject("products", data);      // 傳資料給 Thymeleaf 模板
        return viewModel;
    }
}
```

`productView.html` **(Thymeleaf 模板)**

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<!-- xmlns:th 宣告 Thymeleaf 命名空間, 啟用 th:* 屬性 -->
<head>
    <meta charset="utf-8">
    <title>Product View</title>
    <!-- Bootstrap 4 CDN -->
    <link rel="stylesheet"

href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.0/css/bootstrap.min.css">
</head>
<body>
    <div class="container">
        <h1>Products</h1>
        <div class="row">
            <!-- th:each 迭代商品清單 (對應 JSP 的 c:forEach)  -->
            <div class="col-md-3" th:each="product : ${products}">
                <div class="card">
                    <!-- th:src 動態綁定圖片路徑 -->
                    <img class="card-img-top" th:src="${product.path}"
alt="Product Image">
                    <div class="card-body">
                        <!-- th:text 顯示動態文字 -->
                        <h4 class="card-title"
th:text="${product.productname}">Name</h4>
                        <p  class="card-text"
th:text="${product.description}">Description</p>
                        <p>價格: <span th:text="${product.price}">price</span></p>
                        <!-- th:href 動態組合 URL: @{} 是 Thymeleaf 的 URL 語法 -->
                        <a th:href="@{'cart/buy/' + ${product.id}}" class="btn
btn-primary">Buy</a>
                    </div>
                </div>
            </div>
        </div>
    </div>
</body>
</html>
```

購物車功能

`Item.java` (無 Lombok，標準 JavaBean)

```
public class Item {
    private Product product;
    private int quantity;

    public Item() {}
    public Item(Product product, int quantity) {
        this.product = product;
        this.quantity = quantity;
    }
    // Getter / Setter 省略
}
```

`CartController.java` (Spring Boot 版)

```
import javax.servlet.http.HttpSession;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.ModelAndView;
import com.example.demo.model.*;
import java.util.*;

@RestController
@RequestMapping(value = "/cart")
public class CartController {

    @Autowired
    ProductRepository repository;  // 直接注入 Repository，不需要 ProductModel

    // GET /cart/buy/{id}
    @RequestMapping(value = "/buy/{id}", method = RequestMethod.GET)
    public ModelAndView buy(@PathVariable("id") Integer id, HttpSession session) {
        List<Item> cart;

        if (session.getAttribute("cart") == null) {
            cart = new ArrayList<>();
            Product p = repository.findById(id).get();  // 從 DB 查商品
            cart.add(new Item(p, 1));
            session.setAttribute("cart", cart);
        } else {
            cart = (List<Item>) session.getAttribute("cart");
            int index = this.exists(id, cart);
            if (index == -1) {
                Product p = repository.findById(id).get();
                cart.add(new Item(p, 1));
```

```
            } else {
                int quantity = cart.get(index).getQuantity() + 1;
                cart.get(index).setQuantity(quantity);
            }
            session.setAttribute("cart", cart);
        }

        ModelAndView mv = new ModelAndView("cart");
        mv.addObject("cart", cart);
        return mv;
    }

    private int exists(Integer id, List<Item> cart) {
        for (int i = 0; i < cart.size(); i++) {
            // 注意: Integer 比較用 .equals(), 不可用 ==
            if (cart.get(i).getProduct().getId().equals(id)) {
                return i;
            }
        }
        return -1;
    }
}
```

`cart.html` (Thymeleaf 購物車頁)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Cart Page</title>
    <link rel="stylesheet"

href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.0/css/bootstrap.min.css">
</head>
<body>
    <div class="container">
        <h2>Cart Page</h2>
        <a href="/products">Continue Shopping</a>

        <div class="row">
            <table class="table table-hover" border="1">
                <thead class="table-danger">
                    <tr>
                        <th>Id</th><th>Name</th><th>Photo</th>
                        <th>Price</th><th>Qty</th>
                    </tr>
                </thead>
                <tbody>
                    <!-- th:each 迭代 session 中的購物車 -->
                    <tr th:each="item : ${cart}">
                        <td th:text="${item.product.id}"></td>
```

```
                        <td th:text="${item.product.productname}"></td>
                        <!-- @{} 語法組合路徑，'/' + 變數 -->
                        <td><img th:src="@{'/' + ${item.product.path}}"
width="150"></td>
                        <td th:text="${item.product.price}"></td>
                        <td th:text="${item.quantity}"></td>
                    </tr>
                </tbody>
            </table>
        </div>
    </div>
</body>
</html>
```

兩版本架構比較

面向	Part 1：Spring MVC + Hibernate	Part 2：Spring Boot + JPA
設定方式	XML 設定（web.xml, dispatcher-servlet.xml, hibernate.cfg.xml）	application.properties（自動設定）
視圖技術	JSP + JSTL	Thymeleaf（HTML5 原生相容）
ORM	Hibernate（手動管理 Session/Transaction）	Spring Data JPA（Repository 介面，自動 CRUD）
程式碼量	較多（樣板程式碼多）	較少（Lombok + 自動注入）
部署方式	需要外部 Tomcat，打包成 WAR	內嵌 Tomcat，打包成 JAR，`java -jar` 直接執行
學習曲線	較高（需了解 XML 設定原理）	較低（約定優於設定 Convention over Configuration）

常見錯誤與陷阱

1. `404 Not Found` — 視圖找不到

原因：Controller 回傳的視圖名稱與 ViewResolver 路徑不符。

```
// 錯誤：回傳 "cart/cartindex" 但 JSP 放在 /WEB-INF/jsp/cartindex.jsp
return "cartindex";  // ❌

// 正確：對應 /WEB-INF/jsp/cart/cartindex.jsp
return "cart/cartindex";  // ✅
```

2. 購物車資料消失（Session 遺失）

原因：每次請求都重新建立 `cart` 物件，忘記更新 session。

```
// 修改後必須重新 setAttribute，否則 session 中的資料是舊的
session.setAttribute("cart", cart);  // 不可省略！
```

3. Integer 比較使用 `==`（Spring Boot 版常見）

原因：`Integer` 是包裝類別，`==` 比較記憶體位址，不比較值。

```
// ❌ 錯誤（可能偶爾通過，因為 -128~127 有快取）
if (cart.get(i).getProduct().getId() == id) { ... }

// ✅ 正確：用 .equals() 比較值
if (cart.get(i).getProduct().getId().equals(id)) { ... }
```

4. Hibernate 連線 URL 未指定 `serverTimezone`

原因：MySQL 8 強制要求指定時區，否則拋出 `SQLException`。

```
<!-- ❌ 缺少 serverTimezone -->
jdbc:mysql://localhost:3306/classicmodels

<!-- ✅ 正確 -->
jdbc:mysql://localhost:3306/classicmodels?
serverTimezone=Asia/Taipei&amp;useUnicode=true&amp;characterEncoding=utf8
```

5. JSP 頁面中不使用 `contextPath`

原因：部署路徑不同時，靜態資源路徑會錯誤。

```
<!-- ❌ 硬編碼路徑，部署後可能失效 -->
<img src="/images/p1.jpg">

<!-- ✅ 使用 contextPath，自動對應部署根路徑 -->
<img src="${pageContext.request.contextPath}/images/p1.jpg">
```

練習題

Easy — 基礎理解

1. **`@PathVariable` 用途** 在 `CartController` 的 `buy()` 方法中，`@PathVariable("id")` 的作用是什麼？如果 URL 是 `/cart/buy/p02`，`id` 的值為何？

2. **Session 生命週期** 購物車資料存在 `HttpSession` 中。請問使用者關閉瀏覽器後，購物車資料是否還在？為什麼？

Medium — 功能擴充

3. **加入「清空購物車」功能** 在 `CartController` 新增一個 `clear()` 方法，對應路由 `GET /cart/clear`，執行後清空 session 中的購物車並重導到購物車頁面。

   ▶ 提示
   ▶ 參考解答

4. **購物車商品數量減少** 目前 `buy()` 每次呼叫都是數量 +1。請新增一個 `decrease/{id}` 路由，讓數量 -1，若數量變為 0 則自動從購物車移除。

Hard — 深入挑戰

5. **加入購物車總金額計算（Spring Boot 版）** 目前 `cart.html` 沒有顯示總金額。請在 `CartController.buy()` 中計算總金額，透過 `ModelAndView` 傳給 `cart.html` 並顯示。

6. **防止重複建立 SessionFactory** `ProductModel.from_mvDB()` 每次呼叫都會重建 `SessionFactory`，這在高流量下會耗費大量資源。請思考如何使用 **Singleton 模式** 或 Spring `@Bean` 解決此問題。

學習路線建議

```
基礎                核心                進階                實戰
 |                   |                   |                   |
Java OOP ──► Spring MVC 基礎 ──► Hibernate ORM ──► 完整 CRUD 應用
             (Controller/View)    (Session/Tx)
                    |
                    ▼
             Spring Boot 入門
             (Auto Config)
                    |
                    ▼
             Spring Data JPA
             (Repository)
                    |
                    ▼
             Spring Security
             (登入/權限控制)
```

現在試試看：完成 Part 1 的全部步驟，確認購物車可以加入商品、移除商品，然後嘗試將 ProductModel 改為從 MySQL 讀取資料，感受 Hibernate 帶來的變化。